

1 Блок 1

Решение задачи методом кубической интерполяции

Исходные данные

Требуется найти минимум функции

$$f(x) = \sqrt{1+x^2} + e^{-2x},$$

на интервале

$$[a, b] = [0, 1].$$

Критерий остановки:

$$\varepsilon = 10^{-4}.$$

Производная функции:

$$f'(x) = \frac{x}{\sqrt{1+x^2}} - 2e^{-2x}.$$

Проверим значения производной на концах интервала:

$$f'(0) = -2 < 0,$$

$$f'(1) = \frac{1}{\sqrt{2}} - 2e^{-2} \approx 0.4364 > 0.$$

Следовательно,

$$f'(0) \cdot f'(1) < 0,$$

что позволяет применить метод кубической интерполяции на основе многочлена Эрмита.

Алгоритм метода

В работе используется метод кубической интерполяции на основе интерполяционного многочлена Эрмита. На каждой итерации текущий отрезок обозначается как $[x_0, x_1]$. В концах этого отрезка вычисляются значения функции и производной:

$$y_0 = f(x_0), \quad y_1 = f(x_1),$$

$$y'_0 = f'(x_0), \quad y'_1 = f'(x_1).$$

Обозначим длину текущего отрезка:

$$\Delta = x_1 - x_0.$$

Кубический интерполяционный многочлен Эрмита строится в виде

$$H(x) = H_0(x) + H_1(x),$$

где

$$H_0(x) = [A + B(x - x_0)] \frac{(x_1 - x)^2}{\Delta^2},$$

$$H_1(x) = [C + D(x_1 - x)] \frac{(x - x_0)^2}{\Delta^2}.$$

Коэффициенты вычисляются по формулам:

$$A = y_0, \quad B = y'_0 + \frac{2y_0}{\Delta},$$

$$C = y_1, \quad D = -y'_1 + \frac{2y_1}{\Delta}.$$

Такой многочлен совпадает с исходной функцией и её производной на концах отрезка:

$$H(x_0) = f(x_0), \quad H(x_1) = f(x_1), \quad H'(x_0) = f'(x_0), \quad H'(x_1) = f'(x_1).$$

Затем многочлен представляется в стандартном виде

$$H(x) = \alpha_3 x^3 + \alpha_2 x^2 + \alpha_1 x + \alpha_0.$$

Минимум кубического многочлена находится из условия

$$H'(x) = 0.$$

Так как

$$H(x) = \alpha_3 x^3 + \alpha_2 x^2 + \alpha_1 x + \alpha_0,$$

то

$$H'(x) = 3\alpha_3 x^2 + 2\alpha_2 x + \alpha_1.$$

Следующее приближение x_m находится как корень уравнения

$$3\alpha_3 x^2 + 2\alpha_2 x + \alpha_1 = 0.$$

Если $\alpha_3 \neq 0$, то

$$x_m = \frac{-\alpha_2 \pm \sqrt{\alpha_2^2 - 3\alpha_3 \alpha_1}}{3\alpha_3}.$$

Из двух корней выбирается тот, который лежит внутри текущего интервала:

$$x_m \in (x_0, x_1).$$

Если $\alpha_3 = 0$, то используется формула

$$x_m = -\frac{\alpha_1}{2\alpha_2}.$$

После этого проверяется критерий остановки:

$$|f'(x_m)| < \varepsilon.$$

Если критерий выполнен, то $x^* = x_m$. Если критерий не выполнен, интервал уточняется по знаку производной:

$$\text{если } f'(x_m) < 0, \quad x_0 = x_m,$$

$$\text{если } f'(x_m) > 0, \quad x_1 = x_m.$$

Вычисление коэффициентов на первой итерации

На первой итерации $x_0 = 0$, $x_1 = 1$, $\Delta = 1$. Получаем:

$$y_0 = f(0) = 2, \quad y_1 = f(1) \approx 1.549548846,$$

$$y'_0 = f'(0) = -2, \quad y'_1 = f'(1) \approx 0.436436215.$$

Тогда

$$A = 2, \quad B = -2 + 2 \cdot 2 = 2,$$

$$C \approx 1.549548846, \quad D \approx -0.436436215 + 2 \cdot 1.549548846 = 2.662661476.$$

После раскрытия скобок получаем кубический многочлен

$$H(x) \approx -0.662661476x^3 + 2.212210322x^2 - 2x + 2.$$

Его производная:

$$H'(x) \approx -1.987984428x^2 + 4.424420644x - 2.$$

Корень, лежащий на интервале $(0, 1)$, равен

$$x_m \approx 0.630858.$$

Результаты вычислений

Таблица 1: Последовательность приближений

Итерация	x_0	x_1	x_m	$f'(x_m)$
1	0.000000	1.000000	0.630858	-0.032778
2	0.630858	1.000000	0.650512	0.000785
3	0.630858	0.650512	0.650045	$-7.00 \cdot 10^{-8}$

Так как

$$|f'(0.650045)| < 10^{-4},$$

алгоритм завершает работу.

Полученная точка минимума:

$$x^* = 0.650045.$$

Значение функции в точке минимума:

$$f(x^*) = \sqrt{1 + (0.650045)^2} + e^{-2 \cdot 0.650045} \approx 1.465218.$$

Где формулы используются в коде

Ниже приведены только те фрагменты программы, которые соответствуют расчетным формулам метода кубической интерполяции.

Исходная функция и её производная задаются так:

```
def f(x):  
    return np.sqrt(1 + x**2) + np.exp(-2*x)  
  
def df(x):  
    return x / np.sqrt(1 + x**2) - 2*np.exp(-2*x)
```

Значения y_0 , y_1 , y'_0 , y'_1 и длина отрезка Δ вычисляются так:

```
delta = x1 - x0  
  
y0 = f(x0)  
y1 = f(x1)  
dy0 = df(x0)  
dy1 = df(x1)
```

Коэффициенты A, B, C, D реализованы в коде напрямую по формулам:

```
A = float(y0)  
B = float(dy0 + 2*y0/delta)  
C = float(y1)  
D = float(-dy1 + 2*y1/delta)
```

Построение частей многочлена Эрмита $H_0(x)$, $H_1(x)$ и общего многочлена $H(x)$:

```
p_x = np.poly1d([1.0, 0.0])  
left = p_x - x0  
right = np.poly1d([-1.0, x1])  
  
H0 = np.poly1d(np.polymul(A + B*left, right**2) / delta**2)  
H1 = np.poly1d(np.polymul(C + D*right, left**2) / delta**2)  
  
H = H0 + H1
```

Минимум кубической интерполяции находится как корень производной $H'(x)$:

```
dH = np.polyder(H)  
roots = np.roots(dH)
```

Из корней выбирается тот, который лежит внутри текущего интервала:

```

for root in roots:
    if abs(root.imag) < 1e-10:
        value = float(root.real)
        if x0 < value < x1:
            candidates.append(value)

```

Критерий остановки $|f'(x_m)| < \varepsilon$ и уточнение интервала реализованы так:

```

derivative_value = df(xm)

if abs(derivative_value) < eps:
    return xm, f(xm), np.array(path, dtype=float)

if derivative_value < 0:
    x0 = xm
else:
    x1 = xm

```

Визуализация кубической интерполяции

На каждом графике показываются две кривые: исходная функция $f(x)$ и построенный на текущем отрезке кубический многочлен Эрмита $H(x)$. Точка x_m отмечает минимум кубической интерполяции на данной итерации. Смысл визуализации состоит в том, чтобы показать, как вспомогательный кубический многочлен приближает исходную функцию и как новая точка приближения смещается к истинному минимуму.

Графики

На рисунках ниже представлены исходная функция и её кубическая аппроксимация на нескольких итерациях метода.

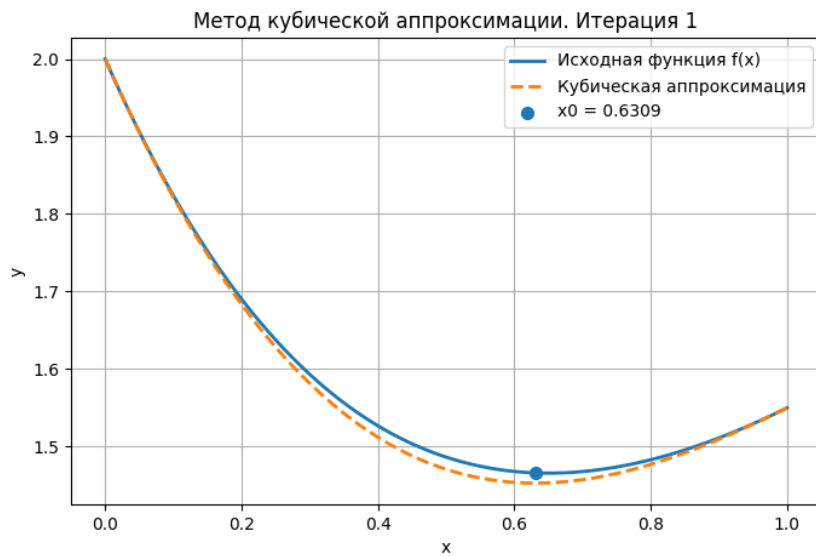


Рис. 1: Первая итерация метода

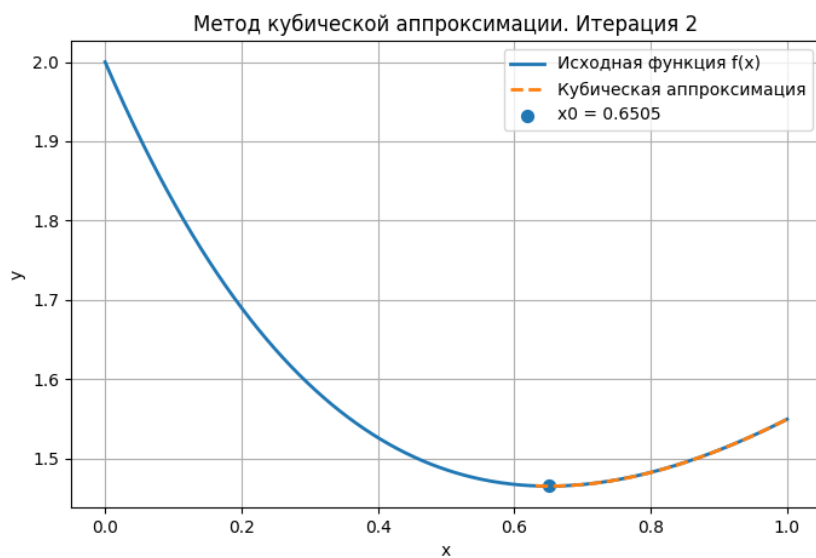


Рис. 2: Вторая итерация метода

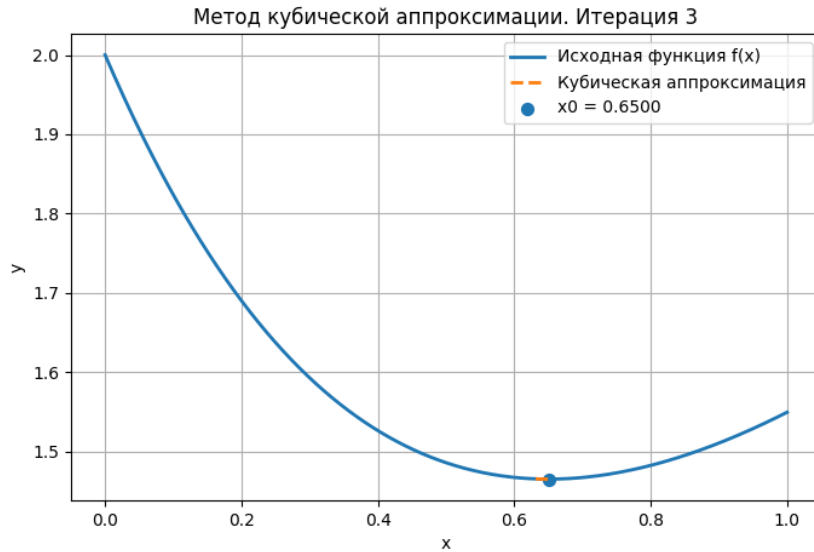


Рис. 3: Третья итерация метода

Вывод

Метод кубической интерполяции на основе многочлена Эрмита позволил определить точку минимума функции

$$f(x) = \sqrt{1+x^2} + e^{-2x}$$

на интервале $[0, 1]$ за три итерации.

Полученный результат:

$$x^* = 0.650045, \quad f(x^*) \approx 1.465218.$$

Требуемая точность

$$\varepsilon = 10^{-4}$$

достигнута.

Блок 2

2 Численные методы безусловной оптимизации функции двух переменных

2.1 Постановка задачи

В этом блоке рассматривается задача минимизации квадратичной функции двух переменных из варианта 7:

$$f(x_1, x_2) = 2x_1^2 + 2x_1x_2 + 3x_2^2 + 10x_1 - 10x_2 + 35.$$

Начальная точка для итерационных методов:

$$X_0 = (2, -3)^T.$$

Критерий остановки:

$$\|\nabla f(X_k)\| < 10^{-4}.$$

Искомая величина - точка минимума

$$X^* = (x_1^*, x_2^*)^T$$

и минимальное значение функции $f(X^*)$. Также требуется построить линии уровня целевой функции и ломаные последовательных приближений для трех методов: покоординатного спуска, градиентного спуска и наискорейшего спуска.

2.2 Аналитическое решение для проверки

Градиент функции:

$$\nabla f(x_1, x_2) = \begin{pmatrix} 4x_1 + 2x_2 + 10 \\ 2x_1 + 6x_2 - 10 \end{pmatrix}.$$

Матрица Гессе:

$$H = \begin{pmatrix} 4 & 2 \\ 2 & 6 \end{pmatrix}.$$

Поскольку

$$\Delta_1 = 4 > 0, \quad \Delta_2 = 4 \cdot 6 - 2 \cdot 2 = 20 > 0,$$

матрица Гессе положительно определена. Следовательно, функция является строго выпуклой и имеет единственный глобальный минимум.

Минимум находится из условия

$$\nabla f(X^*) = 0.$$

То есть решается система

$$\begin{cases} 4x_1 + 2x_2 + 10 = 0, \\ 2x_1 + 6x_2 - 10 = 0. \end{cases}$$

Отсюда

$$x_1^* = -4, \quad x_2^* = 3.$$

Значение функции в этой точке:

$$f(-4, 3) = 0.$$

Таким образом,

$$X^* = (-4, 3)^T, \quad f(X^*) = 0.$$

2.3 Метод покоординатного спуска

В покоординатном спуске на каждом шаге изменяется только одна координата, а остальные временно фиксируются. Для данной квадратичной функции формулы пересчета получаются из частных производных.

При фиксированном x_2 :

$$\frac{\partial f}{\partial x_1} = 4x_1 + 2x_2 + 10 = 0,$$

откуда

$$x_1 = -\frac{2x_2 + 10}{4}.$$

При фиксированном новом x_1 :

$$\frac{\partial f}{\partial x_2} = 2x_1 + 6x_2 - 10 = 0,$$

откуда

$$x_2 = \frac{10 - 2x_1}{6}.$$

В коде эти формулы реализованы в функции `coordinate_descent`:

```
x[0] = -(2*x[1] + 10) / 4
path.append(x.copy())

x[1] = (10 - 2*x[0]) / 6
path.append(x.copy())
```

Промежуточные точки сохраняются после изменения каждой координаты. Поэтому на графике траектория имеет ступенчатый вид: сначала горизонтальный шаг, затем вертикальный.

2.4 Градиентный спуск

Градиентный спуск использует направление антиградиента. Формула метода:

$$X_{k+1} = X_k - \alpha \nabla f(X_k),$$

где α - постоянный шаг. В реализации выбран шаг

$$\alpha = 0.12.$$

В коде формула записана в функции `gradient_descent`:

```
g = grad(x)
x = x - alpha * g
path.append(x.copy())
```

Здесь g - это вектор градиента $\nabla f(X_k)$, а α - длина шага.

2.5 Метод наискорейшего спуска

Метод наискорейшего спуска также движется по антиградиенту, но шаг выбирается не заранее, а оптимально на каждой итерации:

$$X_{k+1} = X_k - \alpha_k \nabla f(X_k).$$

Для квадратичной функции с матрицей Гессе H оптимальный шаг можно найти аналитически:

$$\alpha_k = \frac{g_k^T g_k}{g_k^T H g_k},$$

где

$$g_k = \nabla f(X_k).$$

В коде эта формула реализована в функции `steepest_descent`:

```
g = grad(x)
alpha = (g @ g) / (g @ H @ g)
x = x - alpha * g
path.append(x.copy())
```

2.6 Построение линий уровня

Линия уровня функции - это множество точек, для которых

$$f(x_1, x_2) = C.$$

Чтобы выполнить требование задания, значения уровней берутся не произвольно, а строго по значениям функции в точках траектории:

$$C_k = f(X_k).$$

Поэтому каждая линия уровня проходит через соответствующую вершину ломаной приближений.

В коде это реализовано так:

```
raw_levels = np.array([f(p) for p in path], dtype=float)
levels_through_path = make_levels_from_path(path)
plt.contour(X, Y, Z, levels=levels_through_path, linewidths=1.4)
```

2.7 Направление градиента и линии уровня

Градиент $\nabla f(X_k)$ направлен в сторону наибольшего возрастания функции. Поэтому при минимизации используется противоположное направление:

$$d_k = -\nabla f(X_k).$$

Это направление называется направлением антиградиента. На графиках траектория приближений пересекает линии уровня, переходя от больших значений функции к меньшим.

Для гладкой функции градиент в точке перпендикулярен линии уровня, проходящей через эту точку. Поэтому градиентный спуск и наискорейший спуск на каждом шаге движутся в направлении, которое локально быстрее всего уменьшает значение функции. В градиентном спуске длина шага фиксирована, а в наискорейшем спуске длина шага подбирается заново на каждой итерации.

2.8 Результаты вычислений

Таблица 2: Сравнение методов для функции из ЛР4, вариант 7

Метод	x_1	x_2	$f(x_1, x_2)$	Число точек траектории
Покоординатный спуск	-3.999989	2.999996	≈ 0	17
Градиентный спуск	-3.999974	2.999984	≈ 0	32
Наискорейший спуск	-3.999979	2.999979	≈ 0	15

Все три метода сходятся к аналитическому минимуму $(-4, 3)$. Наискорейший спуск сходится быстрее градиентного спуска с постоянным шагом, так как длина шага пересчитывается на каждой итерации.

2.9 Графики траекторий

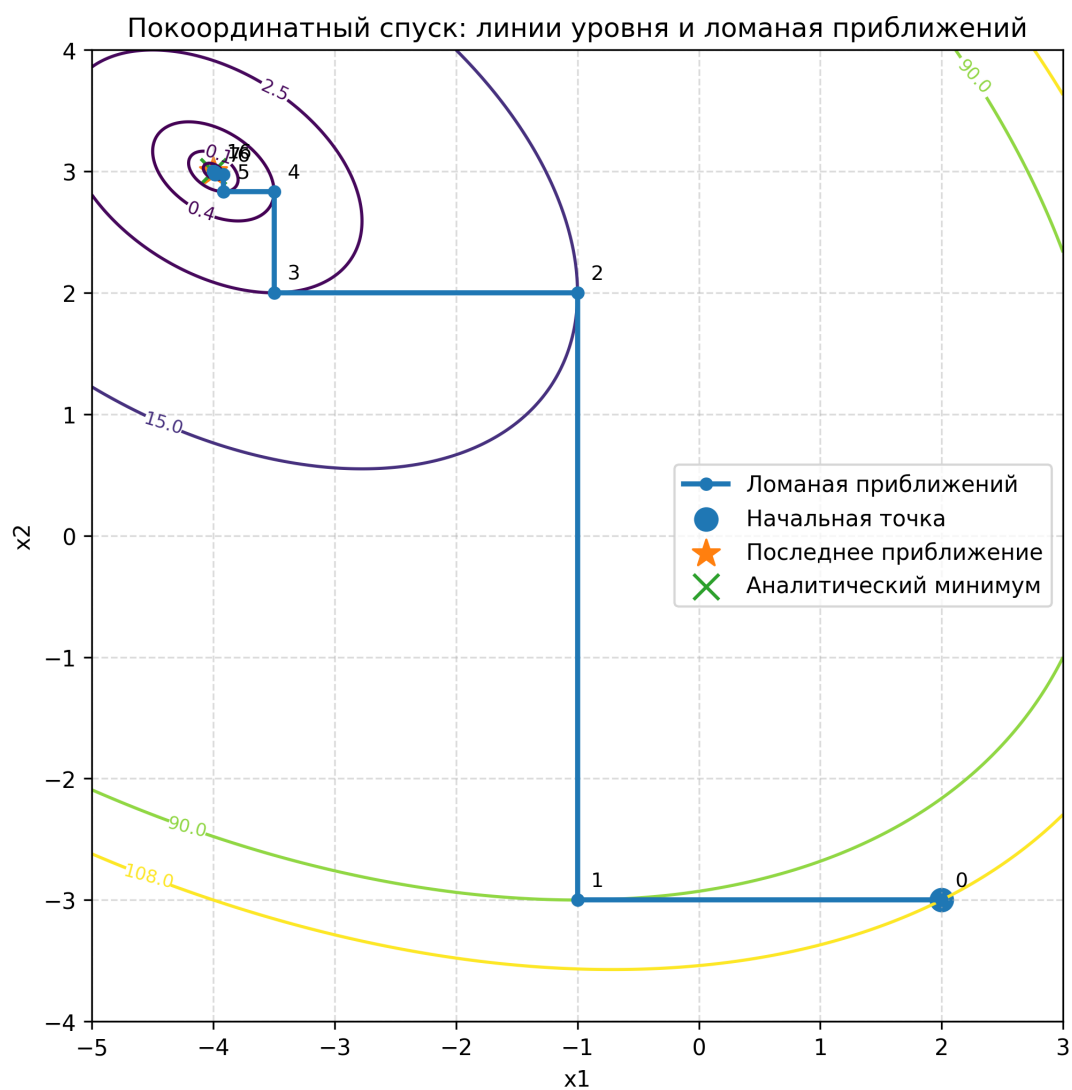


Рис. 4: Покоординатный спуск: ступенчатая ломаная и линии уровня через вершины ломаной

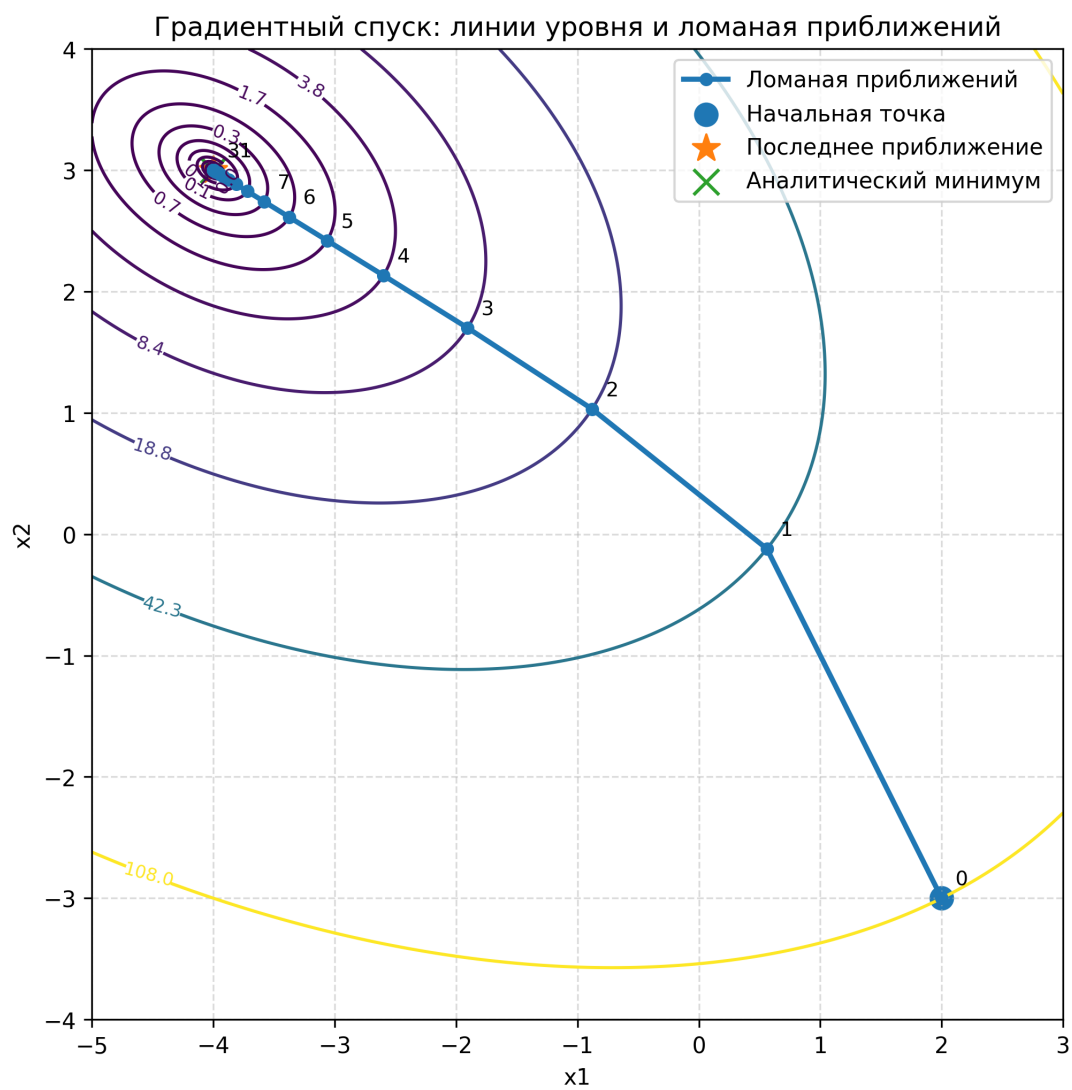


Рис. 5: Градиентный спуск: ломаная приближений и линии уровня через вершины ломаной

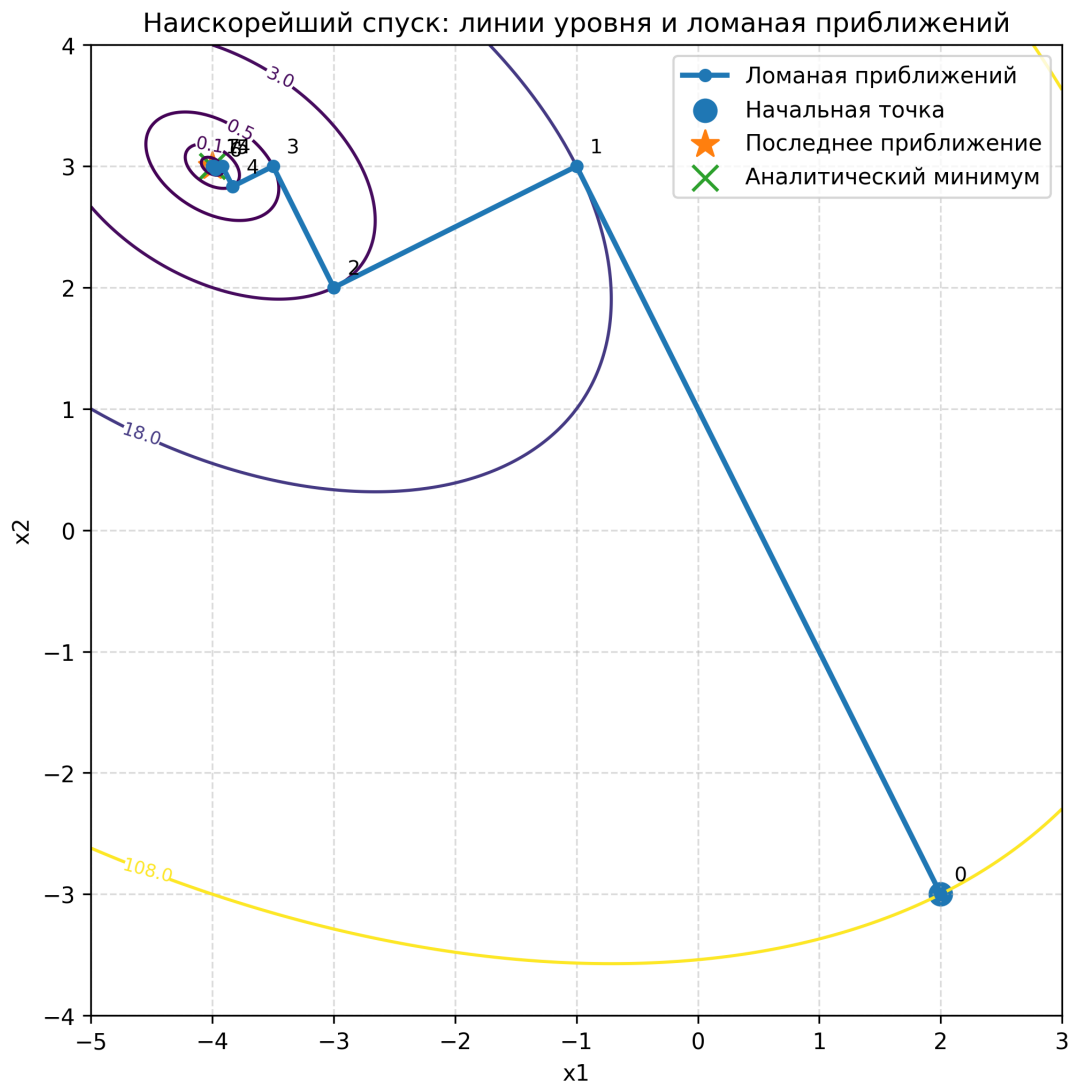


Рис. 6: Наискорейший спуск: ломаная приближений и линии уровня через вершины ломаной

2.10 Метод Ньютона для функции из Д32.2

Для отдельного задания Д32.2 используется функция

$$z(x, y) = x^3 - 3x + y^3 + y^2 - y - 3,$$

начальная точка

$$X_0 = (0.5, 0.5)^T.$$

Градиент:

$$\nabla z(x, y) = \begin{pmatrix} 3x^2 - 3 \\ 3y^2 + 2y - 1 \end{pmatrix}.$$

Матрица Гессе:

$$H_z(x, y) = \begin{pmatrix} 6x & 0 \\ 0 & 6y + 2 \end{pmatrix}.$$

Метод Ньютона имеет вид

$$X_{k+1} = X_k - H_z^{-1}(X_k) \nabla z(X_k).$$

В коде это соответствует решению системы

$$H_z \Delta = -\nabla z(X_k), \quad X_{k+1} = X_k + \Delta.$$

Стационарные точки функции z :

$$\left(1, \frac{1}{3}\right), \quad (1, -1), \quad \left(-1, \frac{1}{3}\right), \quad (-1, -1).$$

По матрице Гессе точка

$$\left(1, \frac{1}{3}\right)$$

является локальным минимумом. Численный метод Ньютона из начальной точки $(0.5, 0.5)$ сходится к этой точке:

$$X^* \approx (1.000000, 0.333333), \quad z(X^*) \approx -5.185185.$$

В программной реализации шаг Ньютона записан как решение линейной системы. Это численно удобнее, чем явно находить обратную матрицу:

```
g = grad(x)
H = hess(x)
delta = np.linalg.solve(H, -g)
x = x + delta
```

Для метода Ньютона также построена визуализация траектории. На графике показаны линии уровня функции $z(x, y)$ и последовательность точек, полученных по формуле Ньютона.

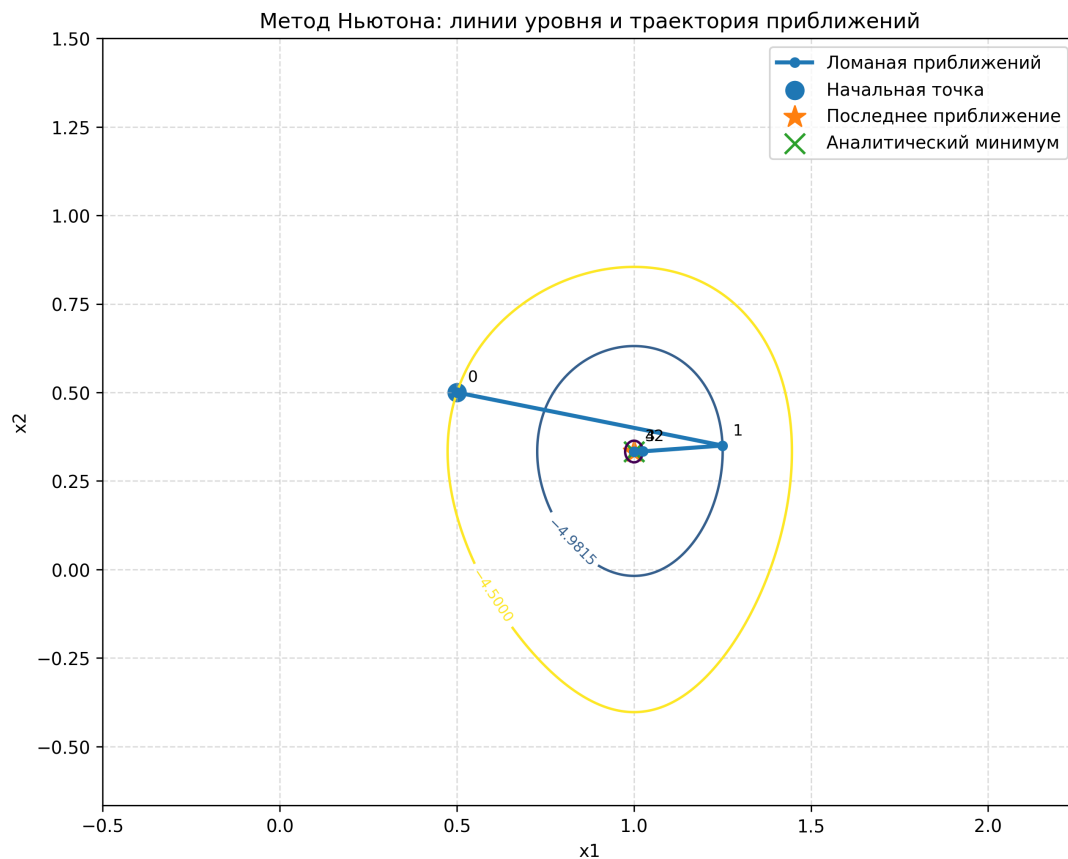


Рис. 7: Метод Ньютона: траектория приближений и линии уровня

2.11 Объектная модель задачи

Для выполнения требования Д32.3 была построена объектная модель задачи минимизации.

Основными сущностями являются:

- `ObjectiveFunction` — класс, описывающий целевую функцию, её градиент и матрицу Гессе;
- `Optimizer` — абстрактный базовый класс для всех методов оптимизации;
- `CoordinateDescent` — класс метода покоординатного спуска;
- `GradientDescent` — класс метода градиентного спуска;
- `SteepestDescent` — класс метода наискорейшего спуска;
- `NewtonMethod` — класс метода Ньютона.

Класс `ObjectiveFunction` хранит математическую постановку задачи:


```
class ObjectiveFunction:
    def __init__(self, func, grad, hess=None):
        self.func = func
        self.grad = grad
        self.hess = hess
```

Базовый класс `Optimizer` задаёт общий алгоритм оптимизации:

```
class Optimizer(ABC):
    def __init__(self, objective, start, eps=EPS, max_iter=1000):
        self.objective = objective
        self.start = np.array(start, dtype=float)
        self.eps = eps
        self.max_iter = max_iter
        self.path = []
```

Здесь используется композиция: каждый оптимизатор содержит объект `ObjectiveFunction`. Также используется агрегация: список `path` хранит траекторию приближений.

Методы `CoordinateDescent`, `GradientDescent`, `SteepestDescent` и `NewtonMethod` наследуются от абстрактного класса `Optimizer` и переопределяют метод `step`. Таким образом реализовано наследование и полиморфизм.

Например, шаг градиентного спуска:

```
class GradientDescent(Optimizer):
    def step(self, x):
        g = self.objective.gradient(x)
        return x - self.alpha * g
```

Шаг метода Ньютона:

```
class NewtonMethod(Optimizer):
    def step(self, x):
        g = self.objective.gradient(x)
        H = self.objective.hessian(x)
        delta = np.linalg.solve(H, -g)
        return x + delta
```

2.12 Вывод по блоку 2

В блоке 2 была исследована квадратичная функция из варианта 7. Аналитически найден минимум $(-4, 3)$, после чего он был подтвержден тремя численными методами. Для каждого метода построена ломаная приближений и линии уровня. Уровни заданы значениями $f(X_k)$, поэтому они проходят через вершины ломаной, что соответствует требованию задания. Также была построена отдельная визуализация метода Ньютона для функции из Д32.2. Также была построена отдельная визуализация метода Ньютона для функции из Д32.2.

Общий вывод

В ходе выполнения работы были рассмотрены численные методы поиска минимума функции одной и двух переменных.

В первом блоке был применён метод кубической интерполяции на основе многочлена Эрмита для функции

$$f(x) = \sqrt{1+x^2} + e^{-2x}$$

на отрезке $[0, 1]$. Перед применением метода была проверена смена знака производной на концах отрезка:

$$f'(0) < 0, \quad f'(1) > 0.$$

Это показывает, что внутри отрезка находится точка минимума.

На каждой итерации строился кубический интерполяционный многочлен, совпадающий с исходной функцией и её производной на концах текущего отрезка. Минимум этого многочлена использовался как новое приближение. Критерий остановки был задан условием

$$|f'(x_m)| < 10^{-4}.$$

В результате метод сошёлся за три итерации. Получена точка минимума

$$x^* = 0.650045,$$

а значение функции в этой точке равно

$$f(x^*) \approx 1.465218.$$

Построенные графики показывают исходную функцию и кубическую интерполяцию на каждой итерации, что позволяет визуально проследить приближение к минимуму.

Во втором блоке была рассмотрена задача безусловной минимизации квадратичной функции двух переменных:

$$f(x_1, x_2) = 2x_1^2 + 2x_1x_2 + 3x_2^2 + 10x_1 - 10x_2 + 35.$$

Аналитически минимум был найден из условия

$$\nabla f(X^*) = 0.$$

В результате получена точка

$$X^* = (-4, 3),$$

при этом

$$f(X^*) = 0.$$

Матрица Гессе данной функции положительно определена, поэтому найденная точка является единственным глобальным минимумом.

Численно минимум был найден тремя методами: покоординатным спуском, градиентным спуском и методом наискорейшего спуска. Покоординатный спуск изменяет координаты по очереди, поэтому его траектория имеет ступенчатый вид. Градиентный спуск движется в направлении антиградиента с постоянным шагом. Метод наискорейшего спуска также использует направление антиградиента, но длина шага на каждой итерации выбирается оптимально. Все три метода сошлись к аналитически найденной точке минимума $(-4, 3)$.

Также был рассмотрен метод Ньютона для функции

$$z(x, y) = x^3 - 3x + y^3 + y^2 - y - 3.$$

Метод Ньютона использует градиент и матрицу Гессе, поэтому учитывает не только направление изменения функции, но и её кривизну. Для данной функции метод сошёлся к точке

$$X^* = \left(1, \frac{1}{3}\right),$$

которая является локальным минимумом.

Для методов второго блока были построены линии уровня и траектории приближений. Линии уровня строились по значениям функции в точках траектории, поэтому они проходят через вершины ломаной приближений. Это позволяет наглядно увидеть направление движения методов к точке минимума и сравнить характер их сходимости.